



The University of Azad Jammu & Kashmir, Muzaffarabad

Computer Architecture & Logic Design

Lab09: Emulator 8086 – Arithmetic Operations

Student Name:

Shahzad Ahmed Awan

Roll No:

2024-SE-15

Course Title:

CL&D

Course Code:

CS-1206

Instructor:

Engr. Sidra Rafique

Submission Date:

August-2025

Department of Software Engineering

Emulator 8086

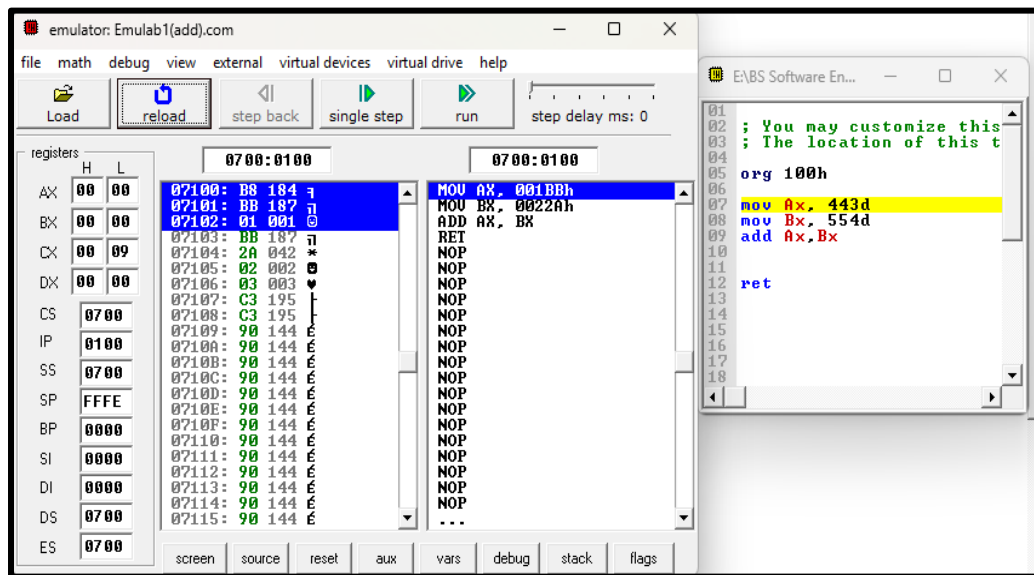
To implement and simulate basic arithmetic operations in x86 assembly language using EMU8086 Emulator, and observe how these operations affect the values stored in CPU registers.

Addition Operation:

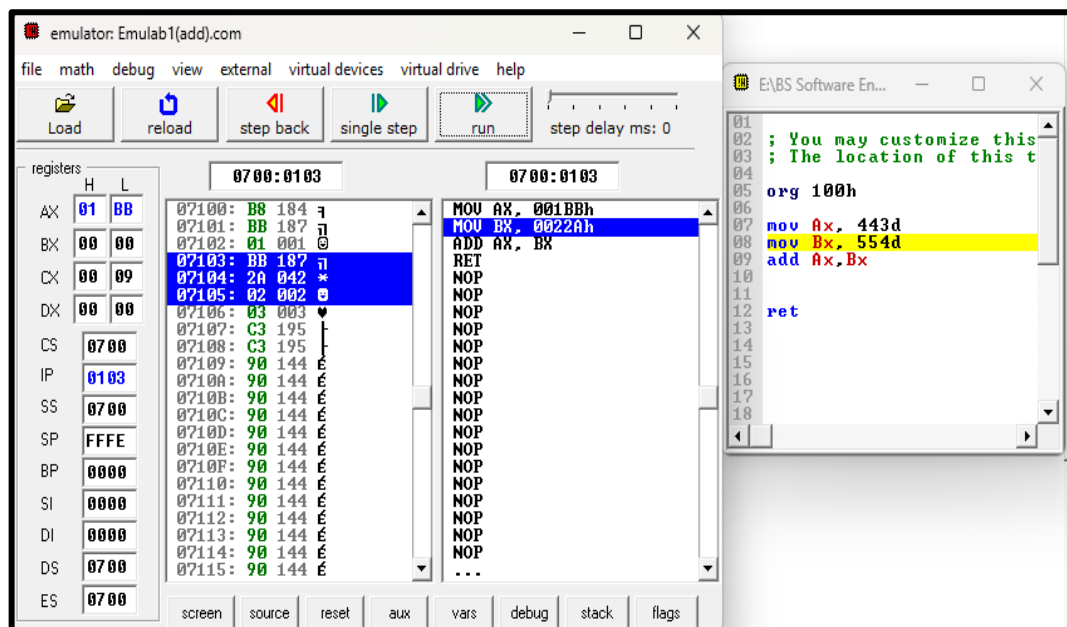
```
05  org 100h
06
07  mov Ax, 443d
08  mov Bx, 554d
09  add Ax, Bx
10
11
12  ret
```

Result:

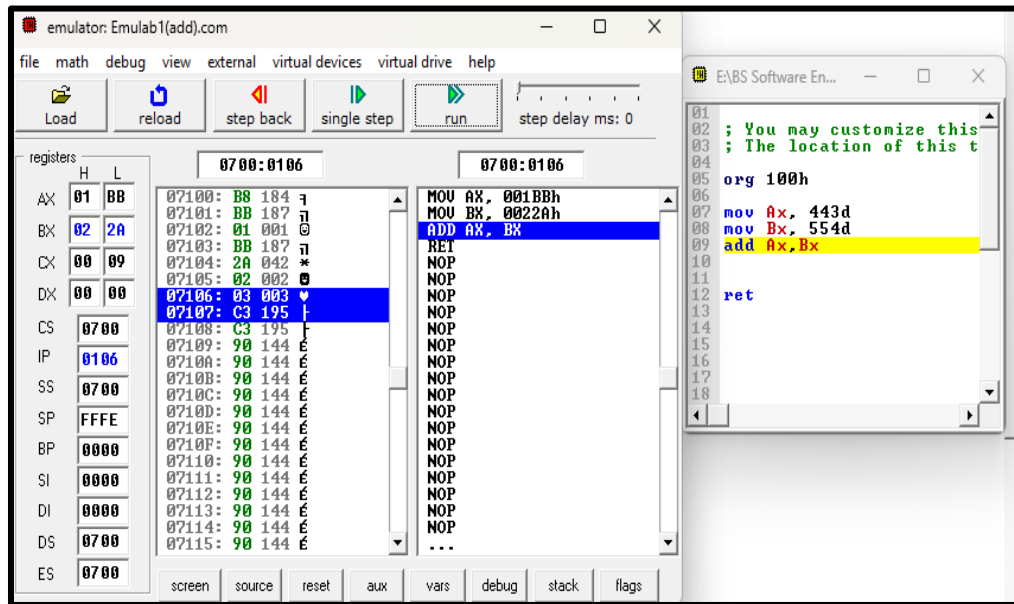
Register	Decimal	Hexadecimal
AX	443	01BB
BX	554	022A



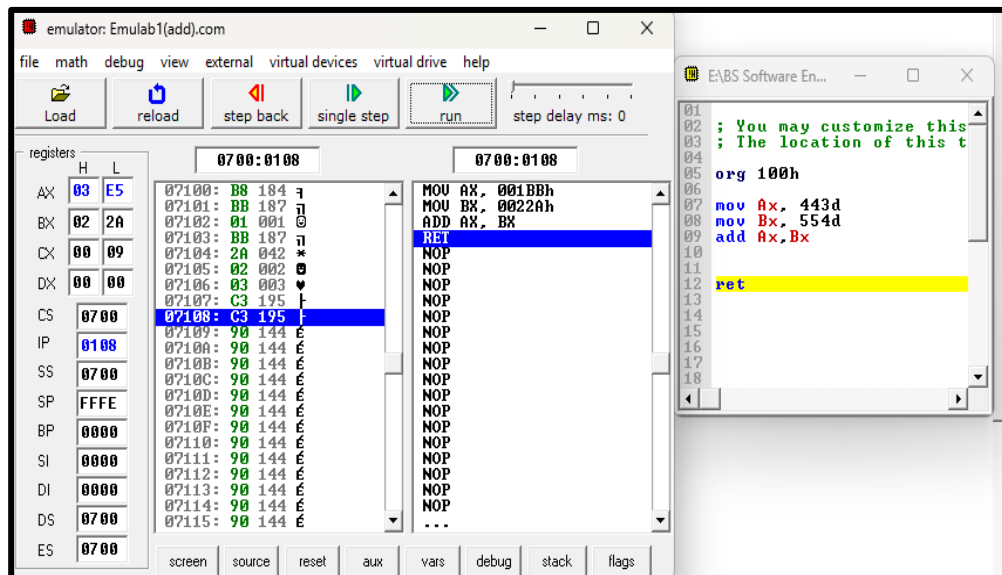
Value Stored in AX Register



Value Stored in BX Register



The Result



Subtraction Operation:

```
04  
05  org 100h  
06  
07  mov Ax, 448d  
08  mov Bx, 552d  
09  sub Ax, Bx  
10  
11  
12  ret  
13
```

The Result in emulator

registers		0700:0108		0700:0108	
	H	L			
AX	FF	98	07100: B8 184		MOV AX, 001C0h
BX	02	28	07101: C0 192		MOV BX, 00228h
CX	00	09	07102: 01 001		SUB AX, BX
DX	00	00	07103: BB 187		RET
CS	0700		07104: 28 040		NOP
			07105: 02 002		NOP
			07106: 2B 043		NOP
			07107: C3 195		NOP
			07108: C3 195		NOP

Multiplication Operation:

```
; You may customize this  
; The location of this t  
  
org 100h  
  
mov ax, 9862d      ; Load  
mov bx, 3427d      ; Load  
mul bx  
  
ret
```

The Result in emulator

registers		0700:0108		0700:0108	
	H	L			
AX	B3	D2	07100: B8 184	↓	MOV AX, 02686h
BX	0D	63	07101: 86 134	↓	MOV BX, 00D63h
CX	00	09	07102: 26 038	&	MUL BX
DX	02	03	07103: BB 187	↓	RET
CS	0700		07104: 63 099	c	NOP
			07105: 0D 013	CRET	NOP
			07106: F7 247	≈	NOP
			07107: E3 227	π	NOP
			07108: C3 195	↓	NOP

Division Operation:

```
org 100h

mov  Ax, 100d
mov  Bx, 20d
div  Ax

ret
```

The Result in emulator

The screenshot displays an x86 emulator window titled "emulator: noname.com_". The assembly code window on the right shows the following code:

```
01 ; You may customize
02 ; The location of the
03
04
05 org 100h
06
07 mov Ax, 100d
08 mov Bx, 20d
09 div Bx
10
11 ret
12
13
14
15
16
17
18
```

The registers window on the left shows the state of the registers:

Register	H	L
AX	00	05
BX	00	14
CX	00	09
DX	00	00
CS	0700	
IP	0108	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	

The memory window shows the instruction at address 07108: C3 195 F, which is the RET instruction. The instruction list on the right shows the execution of the RET instruction, returning control to the address 00064h.